

Table des matières

Problème du plus court chemin	2
Problème	2
Conditions d'existence	3
Exemple	3
Conclusion.....	4
Différents cas	4
Applications.....	5
Variantes du problème	6
Algorithme de Ford.....	7
Principe.....	7
Algorithme de Ford	9
Exemple	10
Remarques	13

PROBLÈME DU PLUS COURT CHEMIN

Problème

Déterminer dans un graphe valué $G=(S, A, V)$ (orienté ou non orienté) le plus court chemin entre deux sommets du graphe.

Les arcs (resp. arêtes) du graphe G sont affectés d'une valeur par une fonction V à valeurs dans R :

$$V : A \rightarrow R$$

$$a \rightarrow V(a)$$

V est appelé la fonction coût (poids, longueur, distance, prix, etc).

On calcule le coût d'un chemin (resp. Chaîne) par la somme des valeurs de ses arcs (resp. Arêtes).

Soit $c=(a_1, a_2, \dots, a_\lambda)$ un chemin de longueur λ dans G

$$\text{coût}(c) = \sum_{a_i \in c} V(a_i)$$

On appelle plus court chemin de s à t un chemin allant de s à t de coût minimum.

Un tel chemin n'existe pas toujours.

Conditions d'existence

Soit c un chemin allant de s à t .

Supposons que ce chemin contient un circuit Γ .

Soit c' le chemin obtenu en supprimant ce circuit de c .

On a $\text{coût}(c) = \text{coût}(c') + \text{coût}(\Gamma)$

Si $\text{coût}(\Gamma)$ est négatif alors

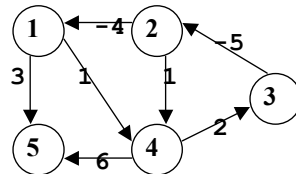
Il n'existe pas de plus court chemin de s à t .

En effet, étant donné un chemin c de s à t , on peut toujours construire un chemin de coût strictement inférieur à c , en passant une fois de plus dans Γ .

Les circuits de coût négatif sont appelés circuits absorbants.

Exemple

On peut diminuer la valeur du chemin allant du sommet 1 au sommet 2 en empruntant le circuit (2 4 3) qui vaut -2



Conclusion

Le plus court chemin de s à t existe : s'il existe un chemin de s à t et qu'il n'y a pas de circuit absorbant.

Autrement dit, si une composante connexe présente un circuit absorbant, alors il n'existe pas de plus court chemin entre deux sommets de cette composante.

Remarque

Un plus court chemin dans G est un plus long chemin dans ${}^T G$.

Différents cas

Soient s et t deux sommets de G .

On distingue 3 cas dans la recherche du plus court chemin entre s à t :

- Il n'existe aucun chemin de s à t (c'est à dire s et t appartiennent à deux composantes connexes différentes de G).
- Il existe un chemin de s à t mais pas de plus court chemin (cas du circuit absorbant)
- Il existe un plus court chemin de s à t .

Applications

- Trouver le moyen le plus économique pour aller de Paris à Dakar, connaissant pour chaque ligne aérienne le prix du billet d'avion.
- Problèmes d'optimisation de réseaux (réseaux routiers ou téléphoniques)
- Etc

Variantes du problème

Le problème de la recherche d'un plus court chemin se rencontre sous l'une des trois formes suivantes :

- On recherche un plus court chemin entre deux sommets donnés.
- On recherche les plus courts chemins entre un sommet donné, appelé source et tous les autres sommets.
- On recherche les plus courts chemins entre tous les sommets pris deux à deux.

Les algorithmes de recherche de plus court chemin sont différents selon les caractéristiques du graphe.

Algorithme de Ford

Principe

Cet algorithme calcul un plus court chemin entre un sommet de départ s et les autres sommets accessibles depuis s dans un graphe sans circuit de valeur strictement négative.

Etapas de l'algorithme

On suppose que tous les sommets sont numérotés de 1 à n : $s_1, s_2, \dots, s_n$.

Cet algorithme utilise deux tableaux :

D = tableau de coûts indicés par les sommets

P = tableau de chemin indicés par les sommets

1) Initialisation

Sommet de départ $\leftarrow s_1$

$D[s_1] \leftarrow 0$

$D[t] \leftarrow \infty \quad \forall t \in S \setminus \{s_1\} = \{s_2, \dots, s_n\}$

$P[s_1] \leftarrow s_1$

$P[t] \leftarrow -1$

2) Traitement des arcs

//Traiter les sommets $s_1, \dots, s_n$ dans cet ordre

Pour chaque sommet s faire

Parcourir l'ensemble $t \in \Gamma(s)$ des successeurs de s de la manière suivante :

$tmp \leftarrow d[s] + \text{coût}(s, t) \quad // t \in \Gamma(s)$

Si $tmp < D[t]$ alors

$D[t] \leftarrow tmp$

$P[t] \leftarrow s$

FinSi

FinPour

3) Test d'arrêt

Répéter l'étape du traitement (2) tant qu'un $D[t]$ aura été modifié

Algorithme de Ford

```
procedure ford(s, G, D, P)
```

```
Début
```

```
// Initialisation
```

```
D[s]  $\leftarrow$  0
```

```
P[s]  $\leftarrow$  s
```

```
Pour t  $\in$  S(G)-{s} faire
```

```
    D[t]  $\leftarrow$   $\infty$ 
```

```
    P[t]  $\leftarrow$  -1
```

```
Finpour
```

```
// Traitement
```

```
Répéter
```

```
    Modif  $\leftarrow$  faux
```

```
    Pour s  $\in$  S(G) faire
```

```
        Pour t  $\in$   $\Gamma(s)$  faire
```

```
            tmp  $\leftarrow$  d[s] + coût(s, t)
```

```
            Si tmp < D[t] alors
```

```
                D[t]  $\leftarrow$  tmp
```

```
                P[t]  $\leftarrow$  s
```

```
            Modif  $\leftarrow$  vrai
```

```
        Finsi
```

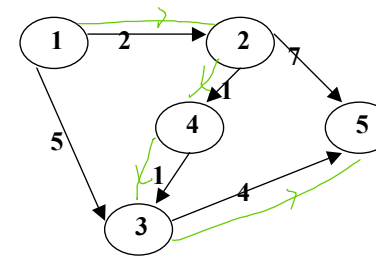
```
    FinPour
```

```
Finpour
```

```
Jusqu'à Modif = faux
```

```
Fin
```

Exemple



Sommet (s)	$\Gamma(s)$
1	{2, 3}
2	{4, 5}
3	{5}
4	{3}
5	{}

Initialisation

D	0	∞	∞	∞	∞
P	1	-1	-1	-1	-1
	1	2	3	4	5

Traitements

- Arcs issus de 1 : $\Gamma(1) = \{2, 3\}$

D	0	2	5	∞	∞
P	1	1	1	-1	-1
	1	2	3	4	5

Modif $\leftarrow$ vrai

- Arcs issus de 2 : $\Gamma(2) = \{4, 5\}$

D	0	2	5	3	9
P	1	1	1	2	2
	1	2	3	4	5

- Arcs issus de **3** : $\Gamma(3)=\{5\}$

D	0	2	5	3	9
P	1	1	1	2	2
	1	2	3	4	5

- Arcs issus de **4** : $\Gamma(4)=\{3\}$

D	0	2	4	3	9
P	1	1	4	2	2
	1	2	3	4	5

- Arcs issus de **5** : $\Gamma(5)=\{\}$

D	0	2	4	3	9
P	1	1	4	2	2
	1	2	3	4	5

- Arcs issus de **1** : $\Gamma(1)=\{2, 3\}$

D	0	2	4	3	9
P	1	1	4	2	2
	1	2	3	4	5

- Arcs issus de **2** : $\Gamma(2)=\{4, 5\}$

D	0	2	4	3	9
P	1	1	4	2	2
	1	2	3	4	5

- Arcs issus de **3** : $\Gamma(3)=\{5\}$

D	0	2	4	3	8
P	1	1	4	2	3
	1	2	3	4	5

Modif ← vrai

- Arcs issus de **4** : $\Gamma(4)=\{3\}$

D	0	2	4	3	8
P	1	1	4	2	3
	1	2	3	4	5

- Arcs issus de **5** : $\Gamma(5)=\{\}$

D	0	2	4	3	8
P	1	1	4	2	3
	1	2	3	4	5

- Arcs issus de **1** : pas de changement dans D et P
- Arcs issus de **2** : pas de changement dans D et P
- Arcs issus de **3** : pas de changement dans D et P
- Arcs issus de **4** : pas de changement dans D et P
- Arcs issus de **5** : pas de changement dans D et P

Remarques

Les d_i ne varient plus $\rightarrow$ elles sont donc les valeurs cherchées.

PCC (s, t) = Plus Court Chemin du sommet s au sommet t

D	0	2	4	3	8
P	1	1	4	2	3
	1	2	3	4	5

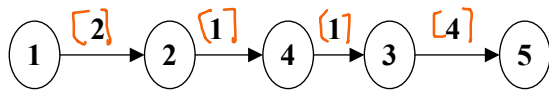
Valeur du PCC (1, 1) = $D[1]=0$ chemin(1, 1) = {}

Valeur du PCC (1, 2) = $D[2]=2$ chemin(1, 2) = 1 - 2

Valeur du PCC (1, 3) = $D[3]=4$ chemin(1, 3) = 1 - 2 - 4 - 3

Valeur du PCC (1, 4) = $D[4]=3$ chemin(1, 4) = 1 - 2 - 4

Valeur du PCC (1, 5) = $D[5]=8$ chemin(1, 5) = 1 - 2 - 4 - 3 - 5



$$D[5] = 2 + 1 + 1 + 4 = 8$$

Remarques

1) $D[t]$ représente le plus court chemin entre le sommet de départ s et les autres sommets t // $t \in S(G) - \{s\}$. En plus, ce résultat est indépendant des signes des valeurs des arcs.

2) Si on change la fonction de comparaison (< par >) dans la condition de l'itération de base càd :

$tmp \leftarrow D[s] + \text{Coût}(s, t)$

si $tmp > D[t]$ Alors

$D[t] \leftarrow tmp$

$P[t] \leftarrow s$

Fsi

D représentera alors le plus long chemin.

3) L'algorithme de Ford est général, mais ne précise pas l'ordre du parcours des arcs du graphe. Le parcours des arcs de G peut se réaliser en visitant tous les sommets de G (dans un ordre donné) et puis en parcourant les arcs incidents vers l'extérieur (ou l'intérieur) de chaque sommet visité.

L'efficacité de cet algorithme dépend de l'ordre de visite des sommets de G (par exemple l'ordre topologique voir TD).